

Slide 1: Growth Functions (Part 1)

Growth functions describe how algorithm running time grows with input size n .

$O(\log n)$: Binary Search - halves search space each step.

$O(n)$: Linear Search - checks each element once.

$O(n \log n)$: Merge Sort/Heap Sort - efficient divide-and-conquer.

Slide 2: Growth Functions (Part 2)

$O(n^2)$: Bubble Sort - nested loops, slow for large inputs.

$O(2^n)$: Recursive Fibonacci - operations double each input.

Order: $O(\log n) \rightarrow O(n) \rightarrow O(n \log n) \rightarrow O(n^2) \rightarrow O(2^n)$

Example for $n=16$:

$O(\log n)=4$, $O(n)=16$, $O(n \log n)=64$, $O(n^2)=256$, $O(2^n)=65536$.

Conclusion: Lower growth functions scale better and are preferred for efficient algorithms.